



Итоговая работа по курсу «Программирование на языке С продвинутый уровень»

Попов Вячеслав Геннадьевич

оглавление

1. Постановка задания
2. Описание программы
 - 2.1 Исходный код программы
3. Описание файлов программы
4. Описание работы программы
 - 4.1 Пошаговый разбор
5. Примеры написания командной строки для запуска
6. Демонстрация работы программы
7. Сборка утилитой *make* (*mingw32-make.exe*)

Постановка задания

Составьте программу, которая находит площадь, образуемую данными уравнениями:

1. $f_1 = 0.6x + 3;$
2. $f_2 = (x - 2)^3 - 1;$
3. $f_3 = \frac{3}{x};$

Требования к программе:

1. Основная программа должна поддерживать опции командной строки, при задании которых:
 - a. Печатаются абсциссы точек пересечения кривых.
 - b. Печатается число итераций, потребовавшихся на приближённое решение уравнений при поиске точек пересечения.
2. Программа должна поддерживать ключ командной строки `--help`, выводящий на печать все допустимые ключи командной строки.
3. Вычисление с точностью ϵ_1 корня x уравнения $f(x) = g(x)$ на отрезке $[a, b]$ должно быть реализовано в отдельной Си-функции `root(f, g, a, b, eps1)`. Если используется метод касательных или комбинированный метод, то у `root` должно быть ещё два параметра функционального типа, позволяющие вызывать производные функций f и g .
4. Вычисление с точностью ϵ_2 величины определённого интеграла от функции $f(x)$ на отрезке $[a, b]$ должно быть реализовано в отдельной Си-функции `integral(f, a, b, eps2)`
5. Си-функции `root` и `integral` должны быть предварительно протестированы. Основная программа должна предоставлять возможности тестирования, активируемые опцией командной строки.
6. Сборка программы должна осуществляться при помощи утилиты `make` (`mingw32-make.exe`). Соответствующий файл должен явно или неявно описывать зависимости между всеми целями сборки. Должны быть определены цели **all** и **clean**, первая из которых полностью собирает программу, а вторая — удаляет все промежуточные файлы (в частности, объектные модули). Сдаваемый архив должен включать в себя Makefile.
7. Программа должна быть снабжена поясняющими комментариями в объёме, достаточном для её понимания. Все глобальные и статические переменные должны быть документированы в комментариях.

Описание программы

Исходный код программы:



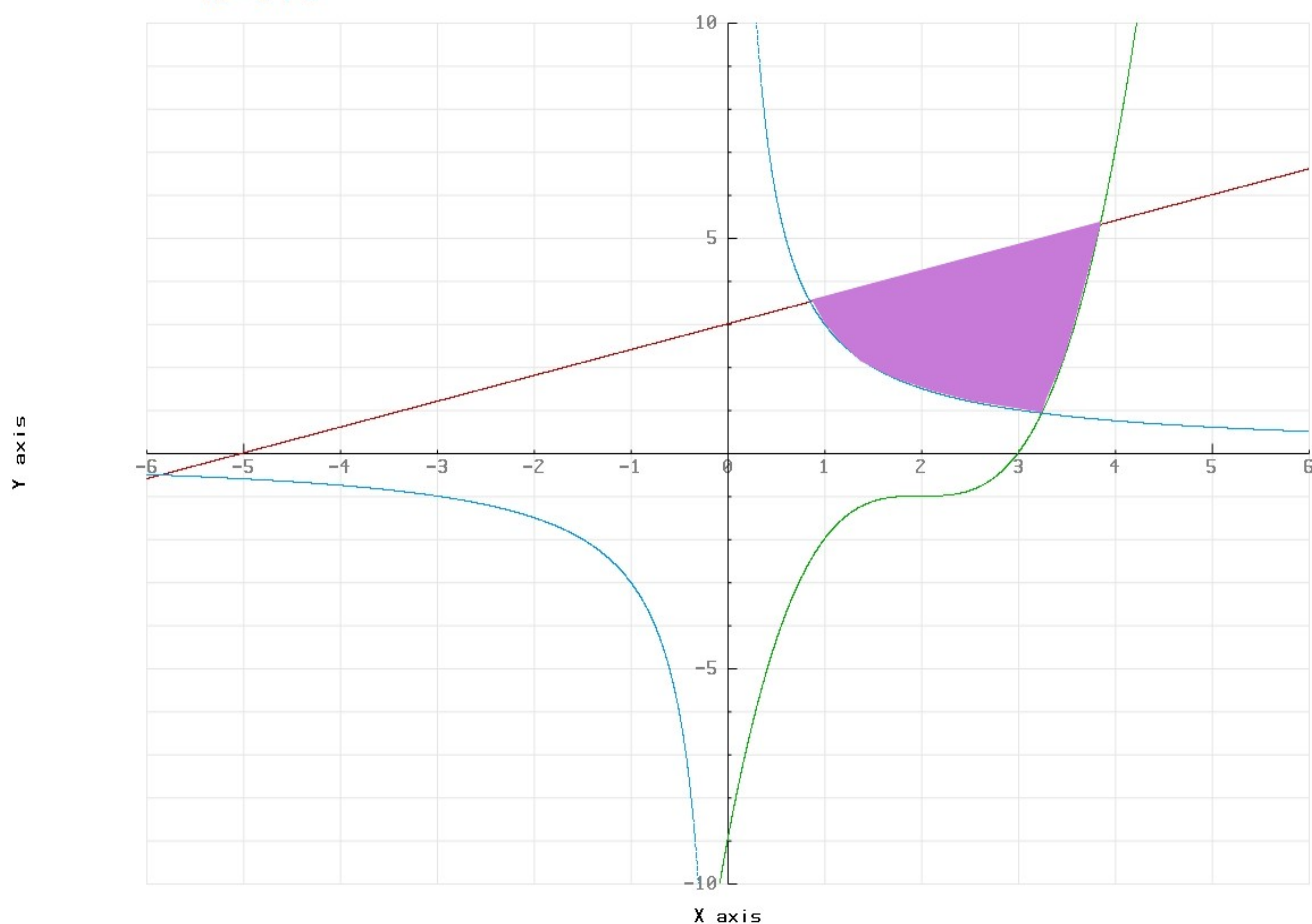
https://github.com/w-popov/c_advanced/tree/main/CWNumericalMethods

Программа *numerical* вычисляет площадь фигуры ограниченной пересечением графиков функций f_1 , f_2 , f_3 путем вычисления интеграла методом трапеций, а поиск корней выполняется методом деления отрезка пополам.

Функции:

$$\begin{aligned}f_1 &= 0.6x + 3; \\f_2 &= (x - 2)^3 - 1; \\f_3 &= \frac{3}{x};\end{aligned}$$

```
f1 = 0.6 * x + 3  
f2 = (x - 2) ^ 3 - 1  
f3 = 3 / x
```



Описание файлов программы

В исходный код программы входят файлы сторонних библиотек:

- [cJSON](#) — два файла: cJSON.h и cJSON.c
- [pbPlots](#) — четыре файла: pbPlots.h, pbPlots.c, supportLib.h, supportLib.c

Библиотека cJSON позволяет работать с форматом json. Программа загружает данные из файла *properties.json* для своей работы. Если файл не существует, то он будет создан с настройками по умолчанию для выполнения задания.

Библиотека pbPlots создает графики функций, изображение графика сохраняется в файл. Имя файла фиксировано как *plot.png*

Файловая структура приложения:

```
.
├── Makefile
├── properties.json
├── README.md
└── src
    ├── cJSON.c
    ├── draw_plots.c
    ├── headers
    │   ├── cJSON.h
    │   ├── draw_plots.h
    │   ├── math_parser.h
    │   ├── numerical_app.h
    │   ├── pbPlots.h
    │   └── supportLib.h
    ├── main.c
    ├── math_parser.c
    ├── numerical_app.c
    ├── pbPlots.c
    └── supportLib.c
```

3 directories, 16 files

- *Makefile* – файл инструкция для сборки утилитой *make* (*mingw32-make.exe*)
- *properties.json* – файл настроек для работы программы. Идея заключается в том, чтобы можно было изменять параметры вычислений без перекомпиляции проекта.

Заголовочные файлы:

- *cJSON.h* – файл заголовков библиотеки cJSON.
- *pbPlots.h* – файл заголовков библиотеки pbPlots.
- *supportLib.h* – файл заголовков библиотеки pbPlots.
- *draw_plots.h* – файл заголовков модуля отрисовки графиков функций.

- *math_parser.h* – файл заголовков модуля парсинга и вычисления математических выражений заданных в файле *properties.json*.
- *numerical_app.h* – файл заголовков модуля вычислений задания.

Файлы с исходным кодом:

- *cJSON.c* – файл исходного кода библиотеки cJSON.
- *pbPlots.c* – файл исходного кода библиотеки pbPlots.
- *supportLib.c* – файл исходного кода библиотеки pbPlots.
- *draw_plots.c* – файл исходного кода модуля отрисовки графиков функций.
- *math_parser.c* – файл исходного кода модуля парсинга и вычисления математических выражений заданных в файле *properties.json*.
- *numerical_app.c* – файл исходного кода модуля вычислений задания.
- *main.c* – файл с точкой входа и функцией `int main(int argc, char *argv[])`

Описание работы программы

Для своей работы программа загружает данные из файла *properties.json*. Если файл не существует, он будет создан автоматически с дефолтными настройками после запуска программы с ключами `-r` и/или `-i`

Структура файла *properties.json*:

```
{
  "inputs": [
    {
      "expr": "0.6 * x + 3",
      "f": 1,
      "start_x": -6.0,
      "end_x": 6.0,
      "num_points": 10000,
      "color": {
        "r": 0.8,
        "g": 0.0,
        "b": 0.0
      }
    },
    {
      "expr": "(x - 2) ^ 3 - 1",
      "f": 2,
      "start_x": -6.0,
      "end_x": 6.0,
      "num_points": 10000,
      "color": {
        "r": 0.0,
        "g": 0.7,
        "b": 0.0
      }
    }
  ]
}
```

```

{
  "expr": "3 / x",
  "f": 3,
  "start_x": -6.0,
  "end_x": 6.0,
  "num_points": 10000,
  "color": {
    "r": 0.0,
    "g": 0.6,
    "b": 1.0
  }
}
],
"roots": [
  {
    "root_expr": "0.6 * x + 3 - 3 / x",
    "F": 1,
    "a": 0.5,
    "b": 1.5
  },
  {
    "root_expr": "(x - 2) ^ 3 - 1 - 3 / x",
    "F": 2,
    "a": 3.0,
    "b": 4.0
  },
  {
    "root_expr": "0.6 * x + 3 - (x - 2) ^ 3 + 1",
    "F": 3,
    "a": 3.5,
    "b": 4.5
  }
],
"integral": [
  {
    "expr_S": "0.6 * x + 3 - 3 / x",
    "from_F": 1,
    "to_F": 2
  },
  {
    "expr_S": "0.6 * x + 3 - (x - 2) ^ 3 + 1",
    "from_F": 2,
    "to_F": 3
  }
],
"Eps1": 0.00001,
"Eps2": 0.00001
}

```

Где:

"inputs" (array) — массив данных графиков функций для отрисовки:

"expr" (str) — выражение формулы для парсинга, вычисления и последующей отрисовки графика

"f" (number) — id функции

"start_x" (number) — координата X начала отрезка для отрисовки

"end_x" (number) — координата X конца отрезка для отрисовки

"num_points" (number) — количество точек на отрисовку графика

"color" (object) — цвет графика rgb

"r" (number) — красный от 0.0 до 1.0

"g" (number) — зеленый от 0.0 до 1.0

"b" (number) — синий от 0.0 до 1.0

"roots" (array) — массив данных для нахождения корней (точек пересечения графиков)

"root_expr" (str) — математическое выражение разности функций

"F" (number) — id корня

"a" (number) — координата X начала поиска корня

"b" (number) — координата X конца поиска корня

"integral" (array) — массив данных для вычисления интеграла

"expr_S" (str) — математическое (подинтегральное) выражение разности функций для вычисления площади n-й фигуры

"from_F" (number) — **id найденного** корня "F" из массива "roots". Начало границы прохода.

"to_F" (number) — **id найденного** корня "F" из массива "roots". Конец границы прохода.

"Eps1": 0.00001 (number) — точность вычислений корней

"Eps2": 0.00001 (number) — точность вычисления интеграла

За основу кода парсера я взял код из статей [url1](#). Код из статей был переписан мной с javascript на C.

Вычисление математических выражений из строки происходит в 3 этапа:

1. Разбиение строки на токены, запись в массив структур `struct Token tokens[MAX_TOKENS]`.
2. Интерпретация токенов алгоритмом «Сортировочная станция», запись в формате обратной польской записи (rpn) в массив структур `struct Token rpn[MAX_TOKENS]`
3. Вычисление обратной польской записи (rpn)

```
const char* expr = "sin(pi / 2) * sqrt(x) - min(5, 10)";
double x_value = 100.0;

struct Token tokens[MAX_TOKENS];
struct Token rpn[MAX_TOKENS];

printf("Входная строка 1: %s\n", expr);
printf("Заданное значение x = %.2f\n\n", x_value);

size_t count = tokenize(expr, tokens);           // (1)
size_t rpn_count = shunting_yard_parse(tokens, count, rpn); // (2)

double result = eval_rpn(rpn, rpn_count, x_value); // (3)
printf("Финальный ответ: %.6lf\n", result);
```

Модуль парсинга поддерживает следующие операторы и операции (*math_parser.h*):

Оператор	Приоритет	Ассоциативность	Действие	Реализация
+	1	левая	сложение	$a + b$
-	1	левая	вычитание	$a - b$
*	2	левая	умножение	$a * b$
/	2	левая	деление	a / b (с проверкой деления на ноль)
^	3	правая	возведение в степень	$\text{pow}(a, b)$

Унарный оператор

Оператор	Приоритет	Действие
~	3	унарный минус (смена знака)

Важно: ~ — это внутреннее обозначение, а не то, что нужно писать в строке. Пользователь вводит обычный -, а токенизатор/«сортировочная станция» распознаёт его как унарный минус.

Скобки и разделители

Символ	Назначение
(	открывающая скобка
)	закрывающая скобка
,	разделитель аргументов функции

Скобки не попадают в RPN — они только управляют порядком выталкивания операторов.

Функции из массива `known_functions`:

С одним аргументом (`args == 1`)

Функция	Действие	Реализация	Ограничение
sin	синус	$\sin(x)$	—
cos	косинус	$\cos(x)$	—
tg	тангенс	$\tan(x)$	—
sqrt	квадратный корень	$\sqrt{x}$	$x \geq 0$ (иначе ошибка)
ln	натуральный логарифм	$\log(x)$	$x > 0$ (иначе ошибка)

С двумя аргументами (`args == 2`)

Функция	Действие	Реализация
max	максимум из двух	$(a > b) ? a : b$
min	минимум из двух	$(a < b) ? a : b$

Синтаксис вызова: $\sin(x)$, $\max(2, 3)$ — обязательно со скобками. Количество аргументов проверяется в сортировочной станции, при несовпадении — ошибка.

Константы из массива `known_constants`:

Константа	Значение
π , PI	$M_PI \approx 3.14159...$
e, E	$M_E \approx 2.71828...$

Переменные

Всё, что не функция и не константа и начинается с буквы/_

После чтения файла *properties.json* программа записывает данные в структуры (*draw_plots.h*):

```
/**
 * Цвет RGB
 */
struct ColorDraw
{
    double r;
    double g;
    double b;
};

/**
 * Массив точек графика
 */
struct ArrayPointsXY
{
    size_t num_points; // кол-во точек
    double *x;         // массив для X
    double *y;         // массив для Y
};

/**
 * Данные о корне
 */
struct Intersection_points
{
    double a;           // начало отрезка
    double b;           // конец отрезка
    double root;        // вычисленный корень
    int id_F;           // id
    wchar_t *root_expr; // выражение для вычисления
    size_t nums_steps;  // кол-во шагов
    size_t rpn_count;
    struct Token rpn[MAX_TOKENS];
};

/**
 * Интеграл
 */
struct IntegralProperties
{
    wchar_t *expr_s; // Формула подинтегральной функции
    int from_f;      // От этого корня
    int to_f;        // До этого
    size_t rpn_count;
    struct Token rpn[MAX_TOKENS];
};
```

```

/**
 * График функции
 */
struct PlotDrawFun
{
    struct ArrayPointsXY points;
    struct ColorDraw color;
    wchar_t *title;
    int fun_number;
};

/**
 * Настройки в JSON файле
 * (Для вычислений)
 */
struct PlotProperties
{
    int f;
    wchar_t *expr;
    double start_x;
    double end_x;
    double intersection_point;
    size_t num_points;
    size_t rpn_count;
    struct ColorDraw color;
    struct Token rpn[MAX_TOKENS];
};

/**
 * Настройки. Главная структура приложения
 */
struct AppProperties
{
    struct PlotDrawFun *pdf;
    struct PlotProperties *plots_props_array;
    struct Intersection_points *inters_points;
    struct IntegralProperties *integral_props_array;
    size_t size_prors_array;
    size_t size_arr_inters_points;
    size_t size_integral_array;
    double eps_1;
    double eps_2;
};

```

Библиотеке pbPlots нужны текстовые данные типа `wchar_t*`, поэтому текстовые выражения хранятся с данным типом.

Пошаговый разбор

Для решения задания нужно сначала заполнить файл *properties.json*. Необходимо в файле *properties.json* заполнить массив "inputs", вписать формулы графиков, границы начала и конца рисования, id, цвета и количество точек. Библиотека *rbPlots* поддерживает интерполяцию. В моем случае я отключил интерполяцию из-за некорректного отображения графика гиперболы при переходе через 0 и других проблем, поэтому рисование графиков функций осуществляется точно и жестко фиксированными пределами осей X, Y.

Далее необходимо заполнить массив "roots", указав выражение разности функций в поле "root_expr", id (обязательно) в поле "F", примерные диапазоны **a** и **b** где пересекаются графики.

И наконец заполнить массив "integral". Т-к площадь фигуры будет находиться как сумма 2 площадей, то заполняется массив двумя объектами: поле "expr_S" — разность верхней и нижней границы ($f_1 - f_3$), для второго объекта верхней границей будет все также f_1 , нижней — f_2 , тогда разность $f_1 - f_2$. Заполняются поля "from_F", "to_F" это значения (id) корней из массива "roots" поля "F". Т-е это границы прохода вычислений. Площадь фигуры вычисляется как сумма площадей 2 фигур из-за переламывания нижней границы (f_3) в точке пересечения f_2 и f_3 .

После запуска программы читается файл *properties.json*, создается и заполняется главная структура приложения:

```
struct AppProperties props = parse_json("properties.json");
props.pdf = make_properties(&props); // сформировать данные для отрисовки графиков
```

Главная структура `struct AppProperties` содержит в себе указатели на созданные динамически массивы структур для вычисления корней и интеграла, отрисовки.

Вычисление корней выполняется функцией: `int root(struct AppProperties *ap)`

Интеграла функций: `int integral(struct AppProperties *ap)`

Отрисовка графиков функцией:

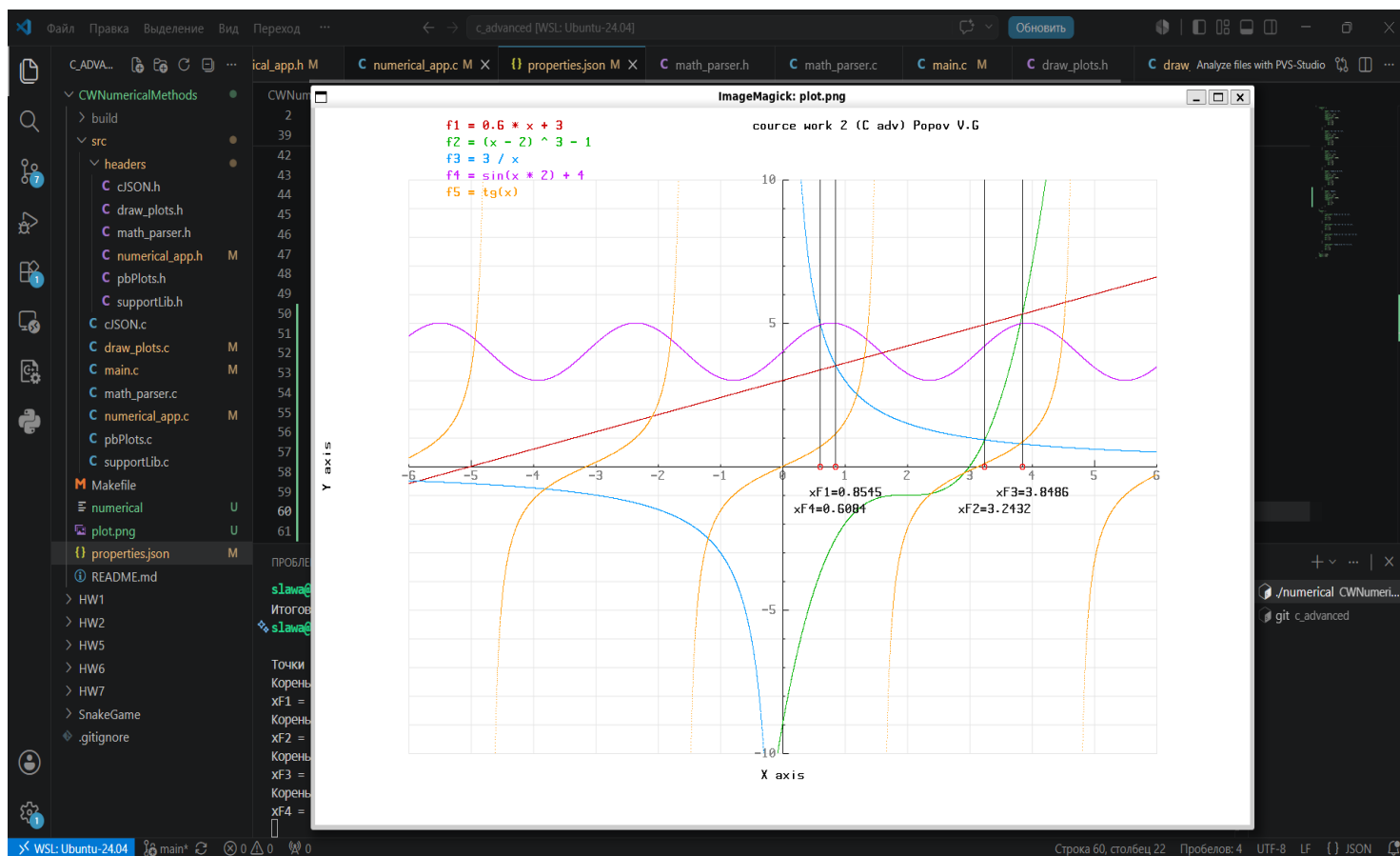
```
void draw_plots(struct AppProperties *ap, size_t nums_draw_plots, const char *filename)
```

При отрисовки графиков, если корни не NaN (т-е уже вычислены), то на холсте рисуются точки найденных корней.

При интерпретации математических выражений, один раз до циклов вычислений строка разбивается на токены, вычисляется RPN алгоритмом «сортировочная станция». RPN выражений хранятся каждые в своей структуре в массивах. Для вычисления точки нужно вызывать только функцию:

```
double eval_rpn(const struct Token* rpn, size_t rpn_count, double x_value)
```

Для отрисовки графиков можно добавить нужное количество объектов в массив `"inputs"`. Для поиска других корней и вычисления площадей, нужно заполнить массивы `"roots"`, `"integral"` в файле `properties.json`:



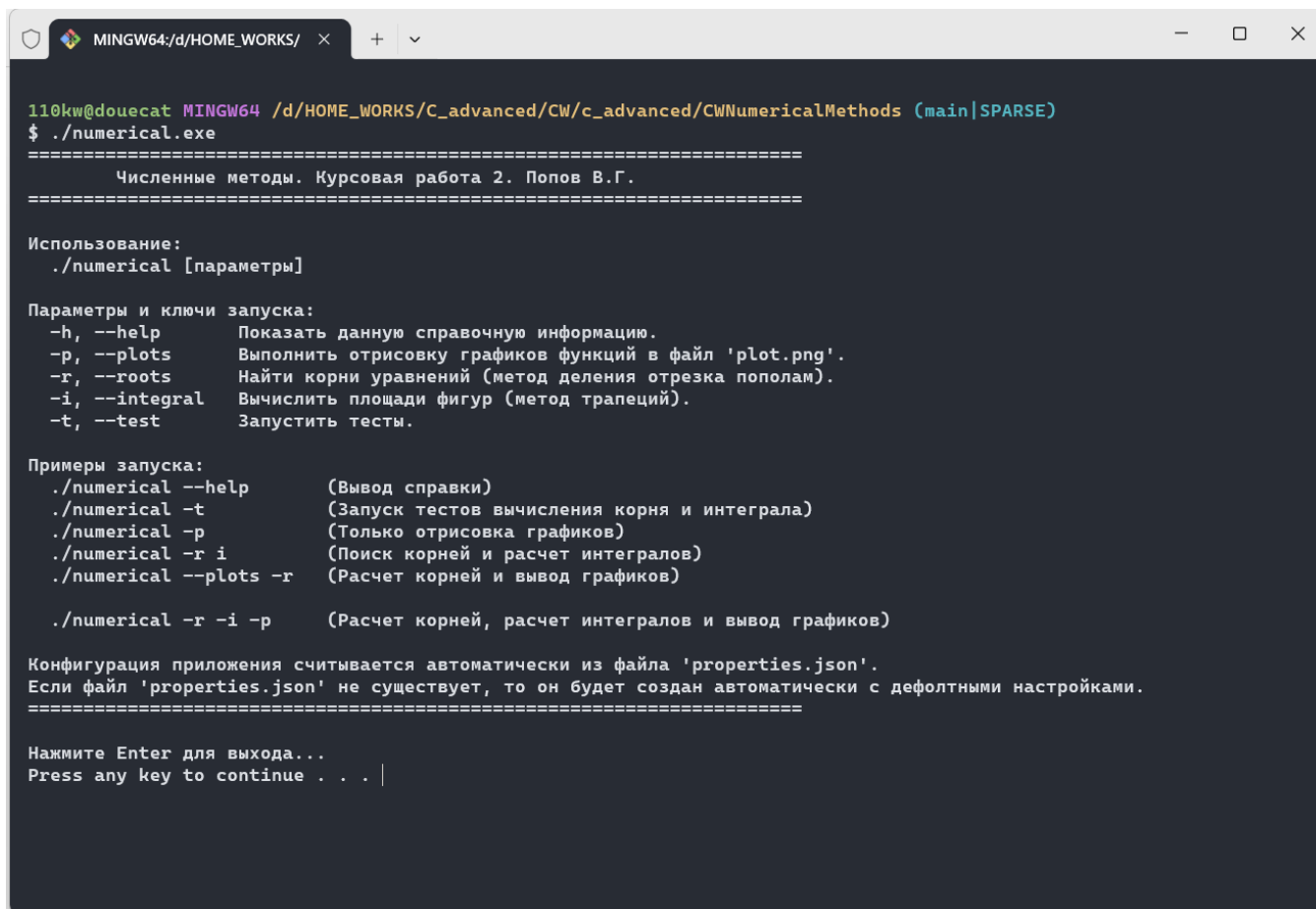
Примеры написания командной строки для запуска

Программа поддерживает следующие ключи для запуска из командной строки:

- -h, --help – Вывод справки
- -p, --plots – Отрисовка графика, сохранение в файл с последующим открытием файла программой заданной в системе по умолчанию для файлов изображений.
- -r, --roots – Вычисление корней
- -i, --integral – Вычисление интеграла
- -t, --test – Запуск тестов

Демонстрация работы программы

рис 1. Запуск программы без аргументов:



```
110kw@douecat MINGW64 /d/HOME_WORKS/ x + v
$ ./numerical.exe
=====
Численные методы. Курсовая работа 2. Попов В.Г.
=====

Использование:
./numerical [параметры]

Параметры и ключи запуска:
-h, --help      Показать данную справочную информацию.
-p, --plots     Выполнить отрисовку графиков функций в файл 'plot.png'.
-r, --roots     Найти корни уравнений (метод деления отрезка пополам).
-i, --integral  Вычислить площади фигур (метод трапеций).
-t, --test      Запустить тесты.

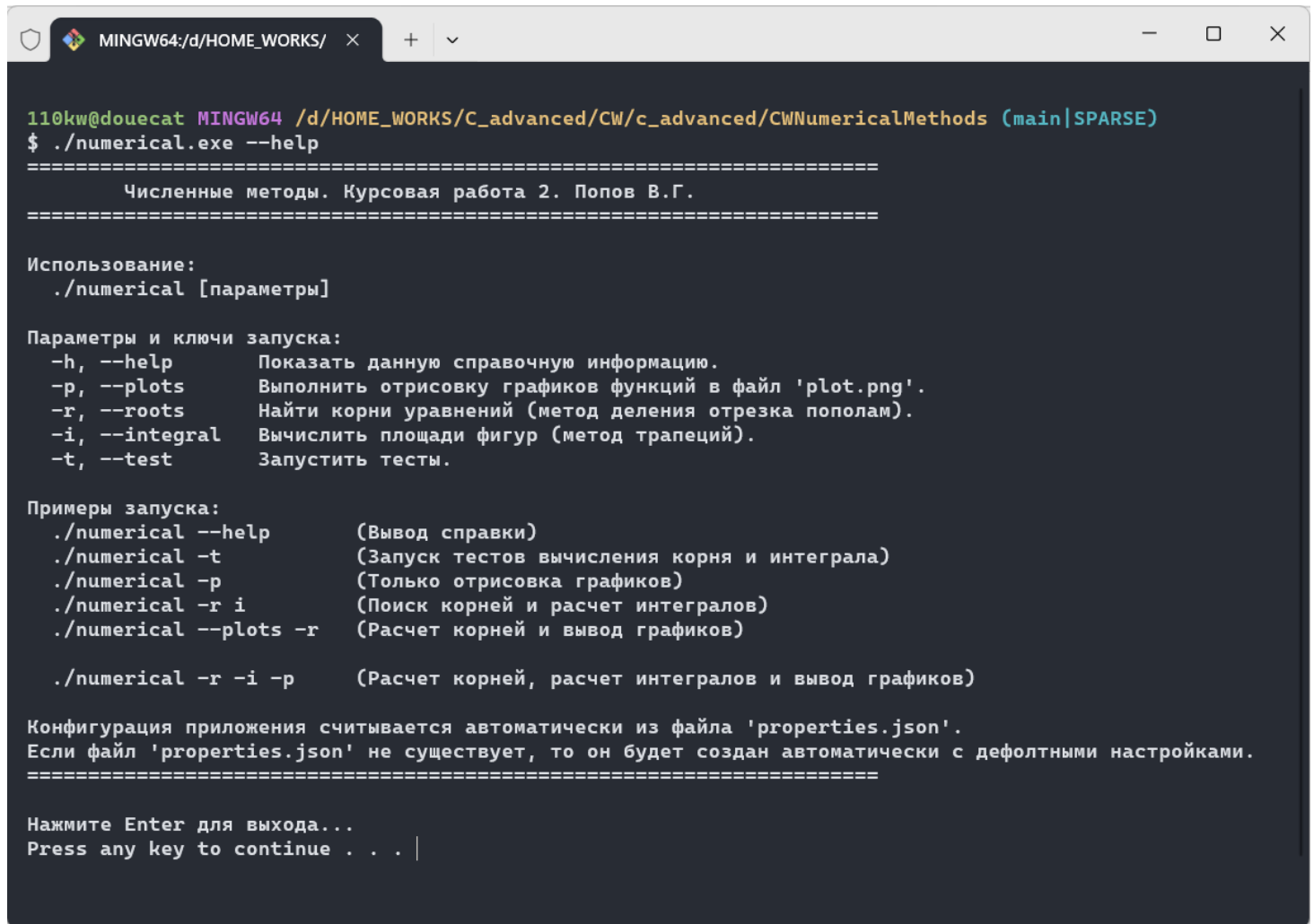
Примеры запуска:
./numerical --help      (Вывод справки)
./numerical -t          (Запуск тестов вычисления корня и интеграла)
./numerical -p          (Только отрисовка графиков)
./numerical -r i        (Поиск корней и расчет интегралов)
./numerical --plots -r  (Расчет корней и вывод графиков)

./numerical -r -i -p    (Расчет корней, расчет интегралов и вывод графиков)

Конфигурация приложения считывается автоматически из файла 'properties.json'.
Если файл 'properties.json' не существует, то он будет создан автоматически с дефолтными настройками.
=====

Нажмите Enter для выхода...
Press any key to continue . . . |
```

рис 2. Запуск программы с ключом -help или -h



```
110kw@douecat MINGW64 /d/HOME_WORKS/ C + v
110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$ ./numerical.exe --help
=====
          Численные методы. Курсовая работа 2. Попов В.Г.
=====

Использование:
./numerical [параметры]

Параметры и ключи запуска:
-h, --help      Показать данную справочную информацию.
-r, --plots     Выполнить отрисовку графиков функций в файл 'plot.png'.
-r, --roots     Найти корни уравнений (метод деления отрезка пополам).
-i, --integral  Вычислить площади фигур (метод трапеций).
-t, --test      Запустить тесты.

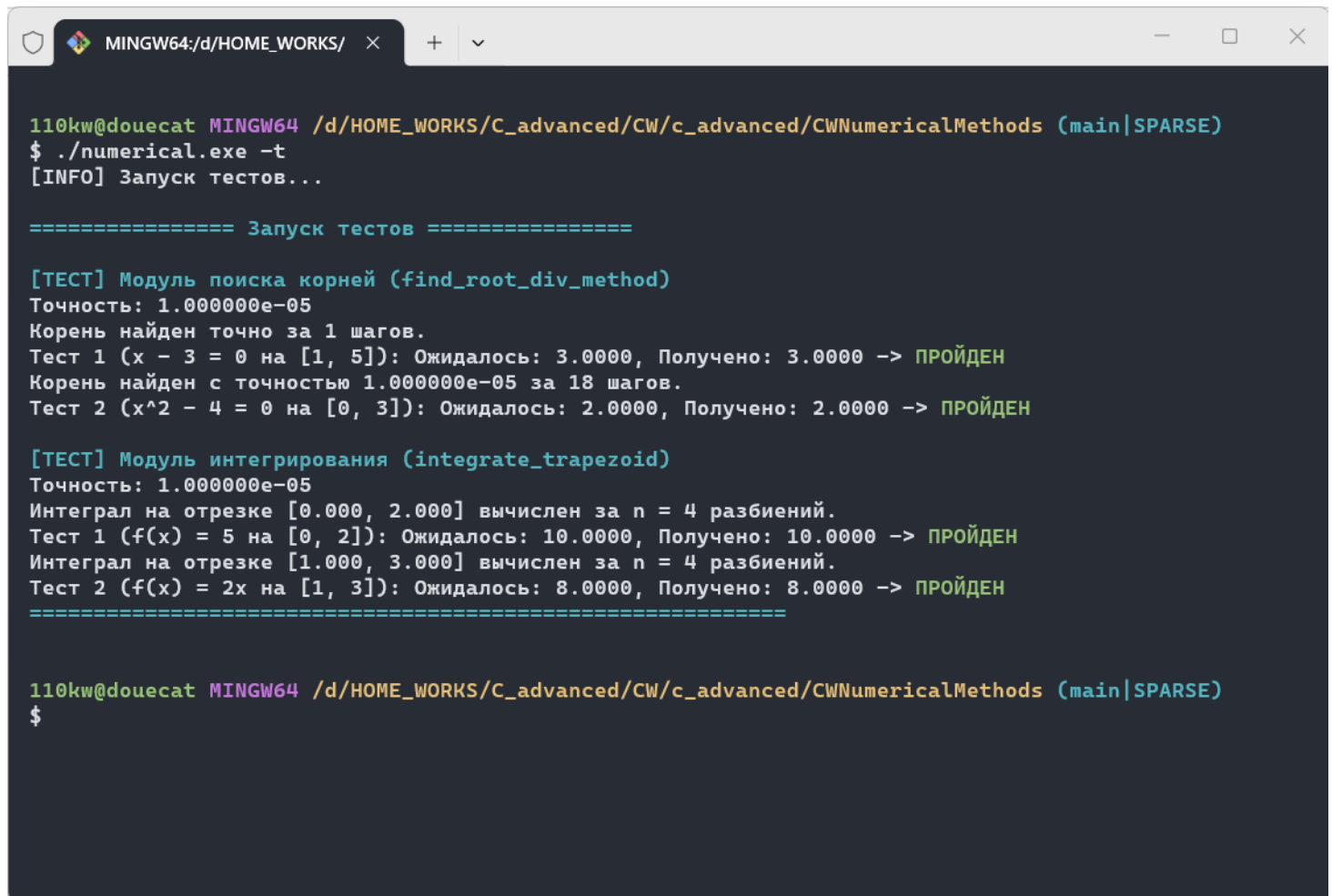
Примеры запуска:
./numerical --help      (Вывод справки)
./numerical -t          (Запуск тестов вычисления корня и интеграла)
./numerical -p          (Только отрисовка графиков)
./numerical -r i        (Поиск корней и расчет интегралов)
./numerical --plots -r  (Расчет корней и вывод графиков)

./numerical -r -i -p    (Расчет корней, расчет интегралов и вывод графиков)

Конфигурация приложения считывается автоматически из файла 'properties.json'.
Если файл 'properties.json' не существует, то он будет создан автоматически с дефолтными настройками.
=====

Нажмите Enter для выхода...
Press any key to continue . . . |
```


рис 3. Запуск программы с ключом -t



```
110kw@douecat MINGW64 /d/HOME_WORKS/ × + ▾  
110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)  
$ ./numerical.exe -t  
[INFO] Запуск тестов...  
  
===== Запуск тестов =====  
  
[ТЕСТ] Модуль поиска корней (find_root_div_method)  
Точность: 1.000000e-05  
Корень найден точно за 1 шагов.  
Тест 1 ( $x - 3 = 0$  на  $[1, 5]$ ): Ожидалось: 3.0000, Получено: 3.0000 -> ПРОЙДЕН  
Корень найден с точностью 1.000000e-05 за 18 шагов.  
Тест 2 ( $x^2 - 4 = 0$  на  $[0, 3]$ ): Ожидалось: 2.0000, Получено: 2.0000 -> ПРОЙДЕН  
  
[ТЕСТ] Модуль интегрирования (integrate_trapezoid)  
Точность: 1.000000e-05  
Интеграл на отрезке  $[0.000, 2.000]$  вычислен за  $n = 4$  разбиений.  
Тест 1 ( $f(x) = 5$  на  $[0, 2]$ ): Ожидалось: 10.0000, Получено: 10.0000 -> ПРОЙДЕН  
Интеграл на отрезке  $[1.000, 3.000]$  вычислен за  $n = 4$  разбиений.  
Тест 2 ( $f(x) = 2x$  на  $[1, 3]$ ): Ожидалось: 8.0000, Получено: 8.0000 -> ПРОЙДЕН  
=====
```

```
110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)  
$
```

рис 4. Запуск программы с ключом -p

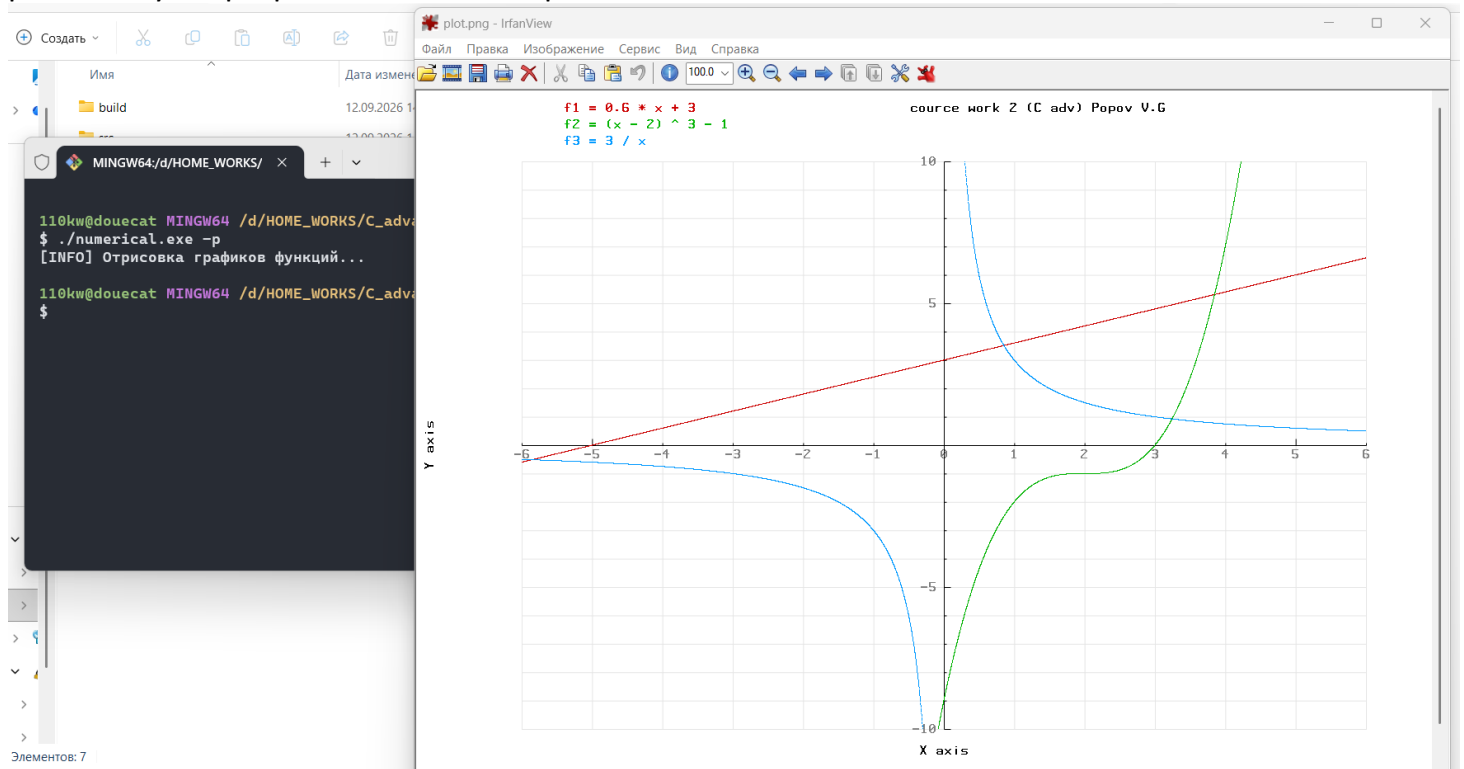


рис 5. Запуск программы с ключом -r

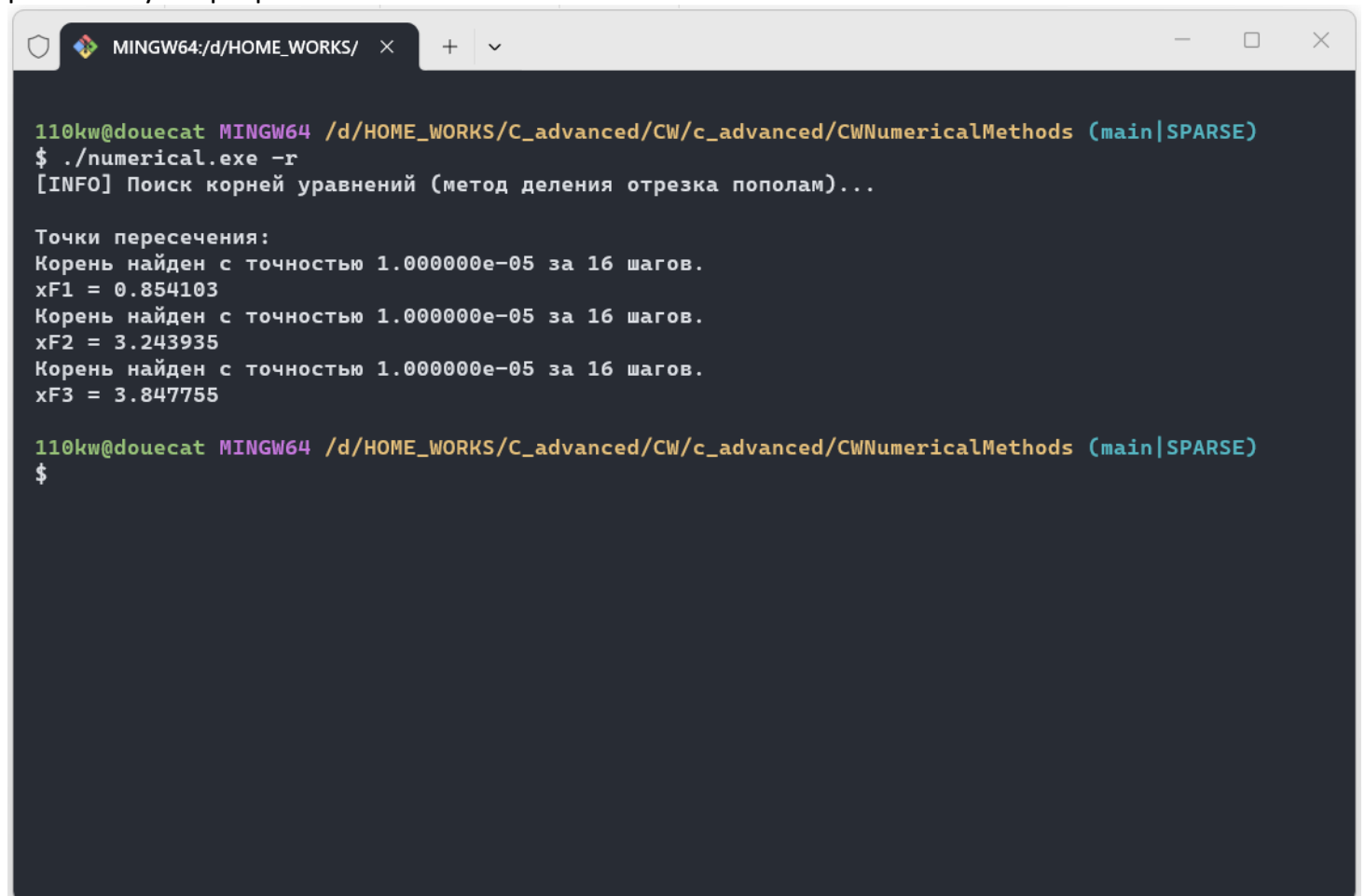


рис 6. Запуск программы с ключом -i

```
MINGW64:/d/HOME_WORKS/ x + v

110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$ ./numerical.exe -i
[INFO] Вычисление интеграла площади (метод трапеций)...

[INFO] Поиск корней уравнений (метод деления отрезка пополам)...

Точки пересечения:
Корень найден с точностью 1.000000e-05 за 16 шагов.
xF1 = 0.854103
Корень найден с точностью 1.000000e-05 за 16 шагов.
xF2 = 3.243935
Корень найден с точностью 1.000000e-05 за 16 шагов.
xF3 = 3.847755
Интеграл на отрезке [0.854, 3.244] вычислен за n = 1024 разбиений.
Площадь 1 участка: 6.104108
Интеграл на отрезке [3.244, 3.848] вычислен за n = 256 разбиений.
Площадь 2 участка: 1.384314
Итоговая площадь всей фигуры: 7.488422
Точность: 1.000000e-05

110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$
```

рис 7. Запуск программы с ключами -i -p

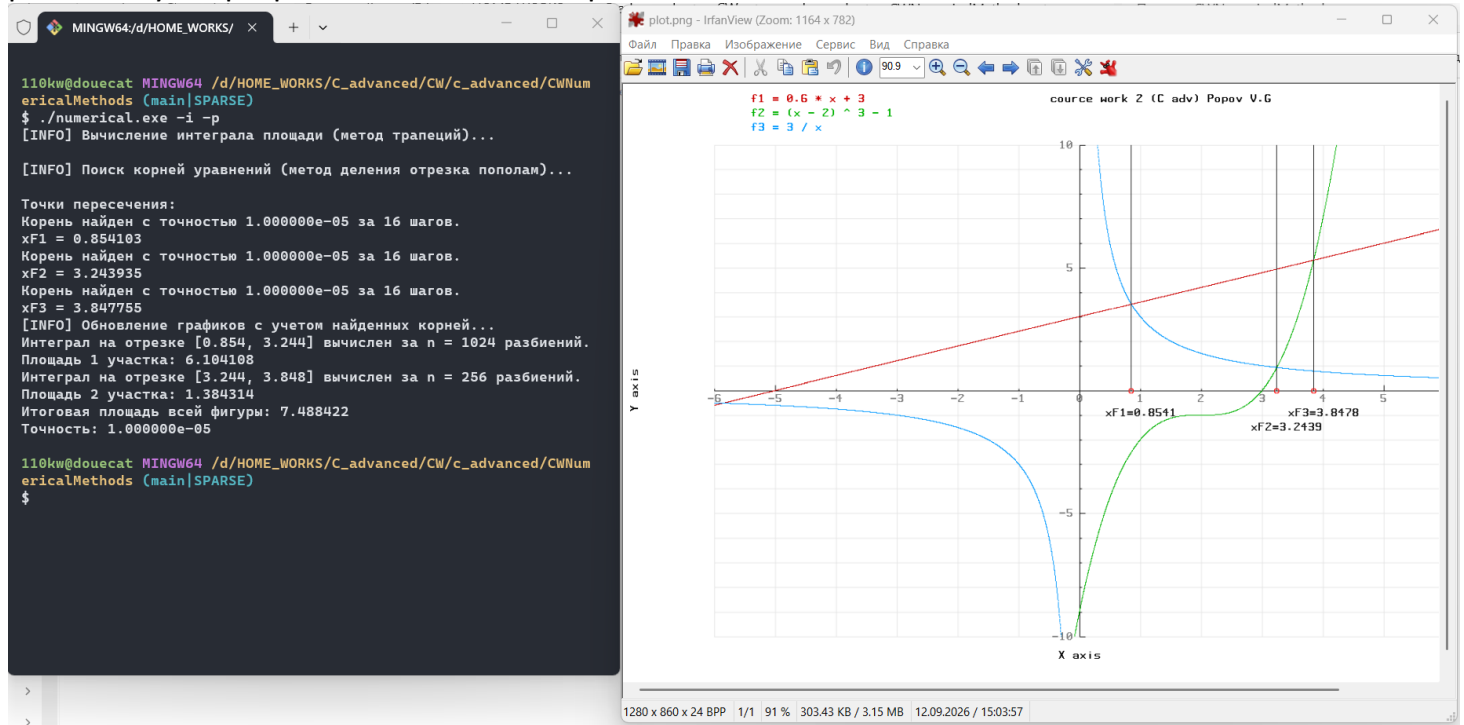
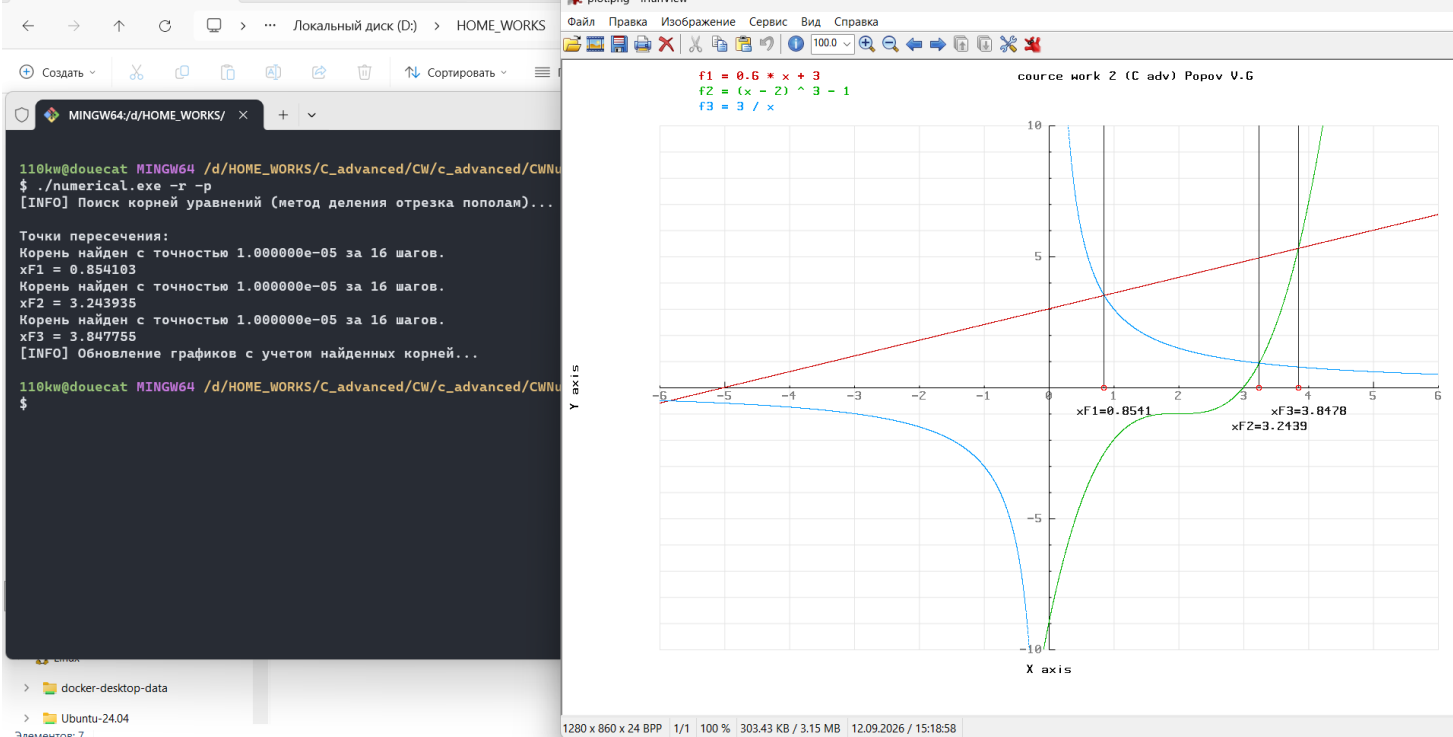


рис 8. Запуск программы с ключами -r -p



Сборка утилитой *make* (*mingw32-make.exe*)

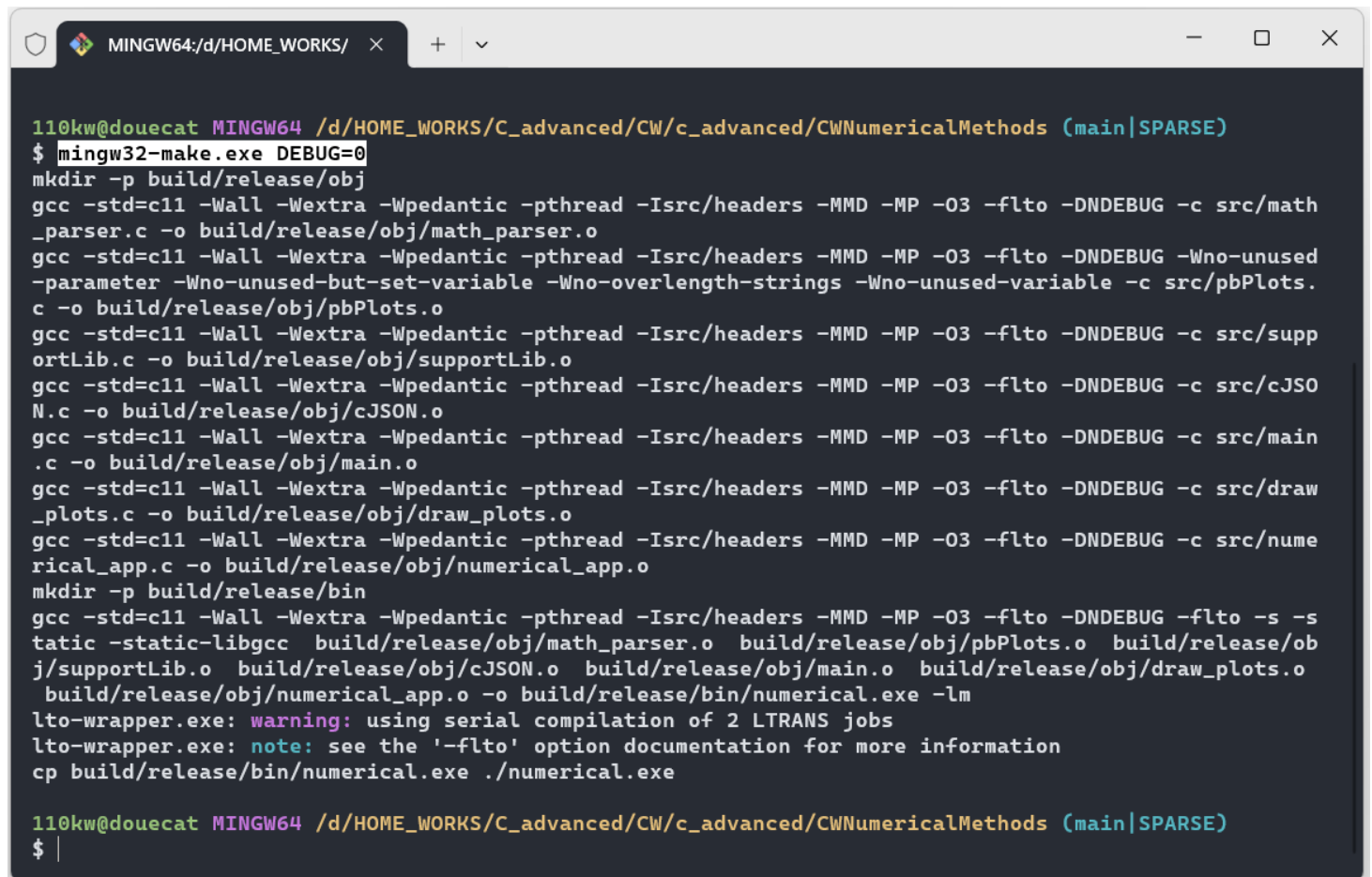
Программа собирается в 2 режимах: debug или release.

Сборка в режиме debug: `mingw32-make.exe`

Сборка в режиме release: `mingw32-make.exe DEBUG=0`

Программа собранная в режиме release является переносимой, т-к компонуется с библиотекой `libgcc` статически. Также в этом режиме происходит компиляция с включенными флагами максимальной оптимизации, удаления отладочной информации.

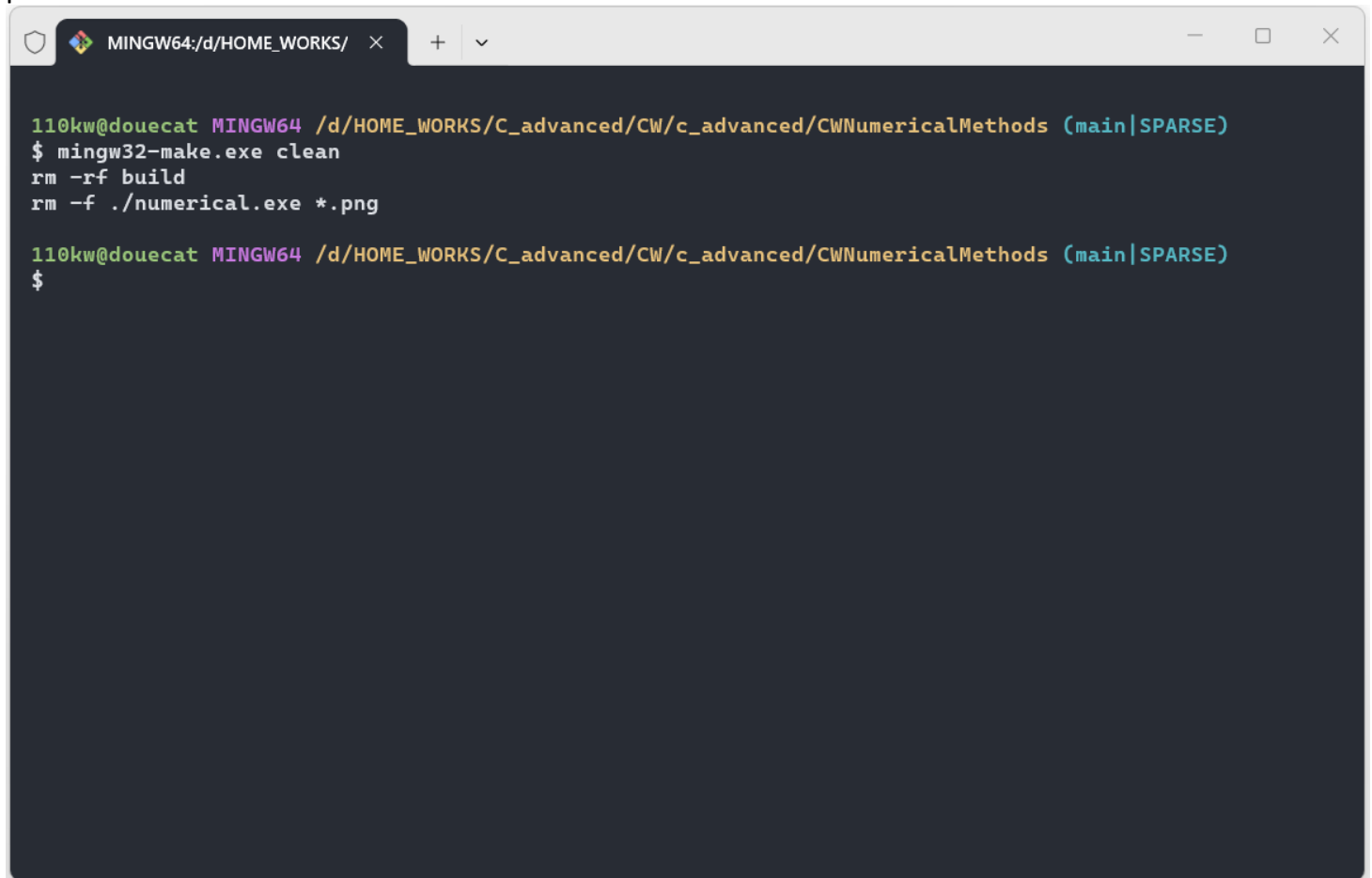
рис 9. Сборка



```
MINGW64:/d/HOME_WORKS/ x + v
110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$ mingw32-make.exe DEBUG=0
mkdir -p build/release/obj
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/math_parser.c -o build/release/obj/math_parser.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -Wno-unused-parameter -Wno-unused-but-set-variable -Wno-overlength-strings -Wno-unused-variable -c src/pbPlots.c -o build/release/obj/pbPlots.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/supportLib.c -o build/release/obj/supportLib.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/cJSON.c -o build/release/obj/cJSON.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/main.c -o build/release/obj/main.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/draw_plots.c -o build/release/obj/draw_plots.o
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -c src/numerical_app.c -o build/release/obj/numerical_app.o
mkdir -p build/release/bin
gcc -std=c11 -Wall -Wextra -Wpedantic -pthread -Isrc/headers -MMD -MP -O3 -flto -DNDEBUG -flto -s -static -static-libgcc build/release/obj/math_parser.o build/release/obj/pbPlots.o build/release/obj/supportLib.o build/release/obj/cJSON.o build/release/obj/main.o build/release/obj/draw_plots.o build/release/obj/numerical_app.o -o build/release/bin/numerical.exe -lm
lto-wrapper.exe: warning: using serial compilation of 2 LTRANS jobs
lto-wrapper.exe: note: see the '-flto' option documentation for more information
cp build/release/bin/numerical.exe ./numerical.exe

110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$ |
```

рис 10. Очистка



```
MINGW64:/d/HOME_WORKS/ × + v
110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$ mingw32-make.exe clean
rm -rf build
rm -f ./numerical.exe *.png

110kw@douecat MINGW64 /d/HOME_WORKS/C_advanced/CW/c_advanced/CWNumericalMethods (main|SPARSE)
$
```